COMP2271: Computer Graphics

Part 6: Advanced Rendering Techniques

Dr. Frederick Li

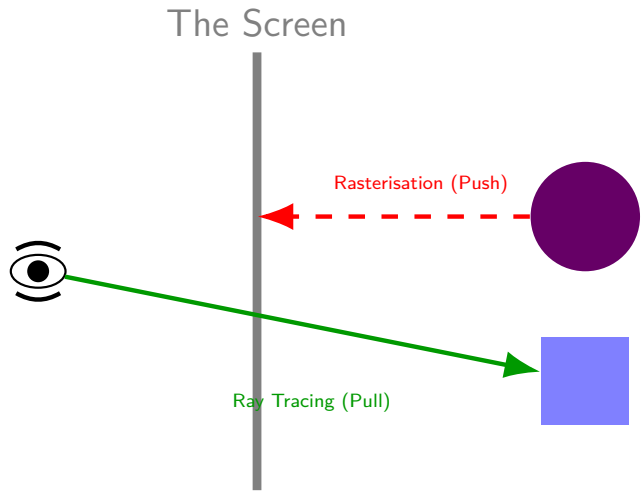
Department of Computer Science
Durham University

Outline

The Plato's Cave of Graphics

The Fundamental Illusion

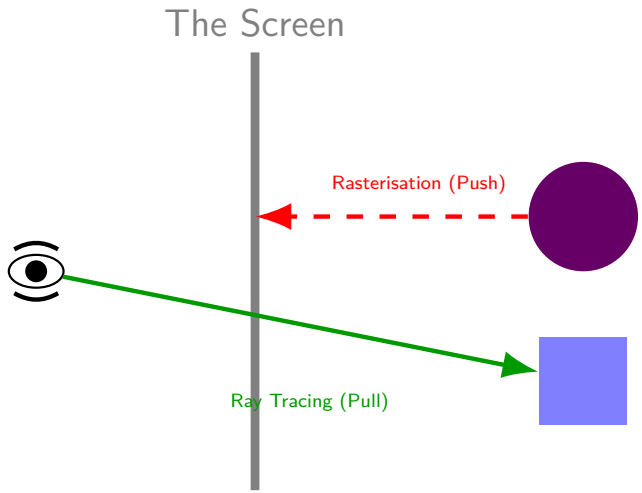
In philosophy, Plato described prisoners chained in a cave, seeing only shadows of objects cast on a wall. They believed the shadows were the reality.



The Plato's Cave of Graphics

The Fundamental Illusion

In philosophy, Plato described prisoners chained in a cave, seeing only shadows of objects cast on a wall. They believed the shadows were the reality.
Computer Graphics is exactly this cave.

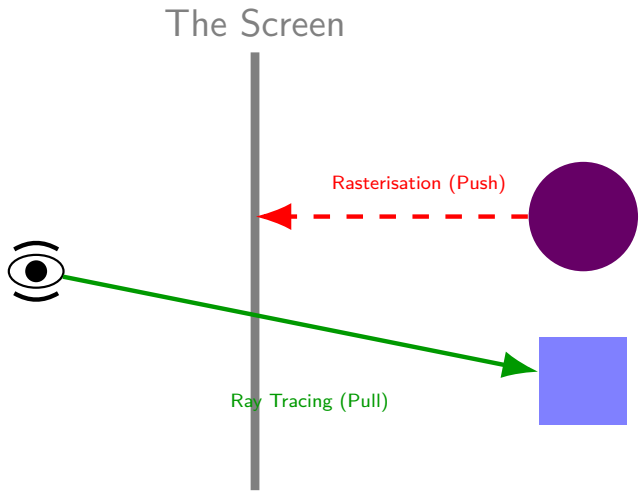


The Plato's Cave of Graphics

The Fundamental Illusion

In philosophy, Plato described prisoners chained in a cave, seeing only shadows of objects cast on a wall. They believed the shadows were the reality.
Computer Graphics is exactly this cave.

- Until now, we have been "Rasterisers". We take a 3D reality and forcefully flatten it onto a 2D wall (the screen).

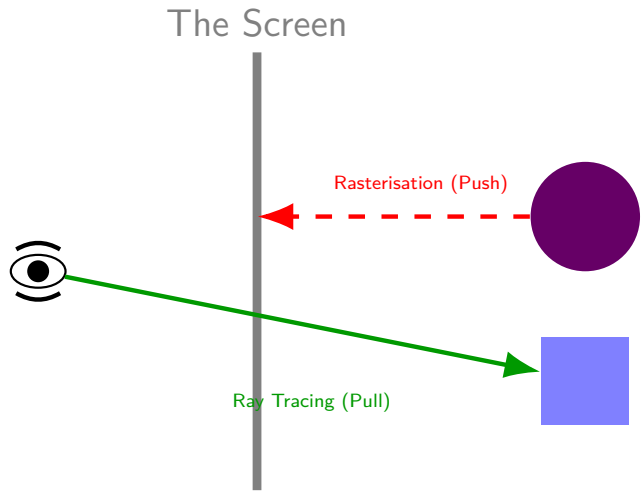


The Plato's Cave of Graphics

The Fundamental Illusion

In philosophy, Plato described prisoners chained in a cave, seeing only shadows of objects cast on a wall. They believed the shadows were the reality.
Computer Graphics is exactly this cave.

- Until now, we have been "Rasterisers". We take a 3D reality and forcefully flatten it onto a 2D wall (the screen).
- But what if, instead of looking at the wall, we turned around and looked at the light itself?

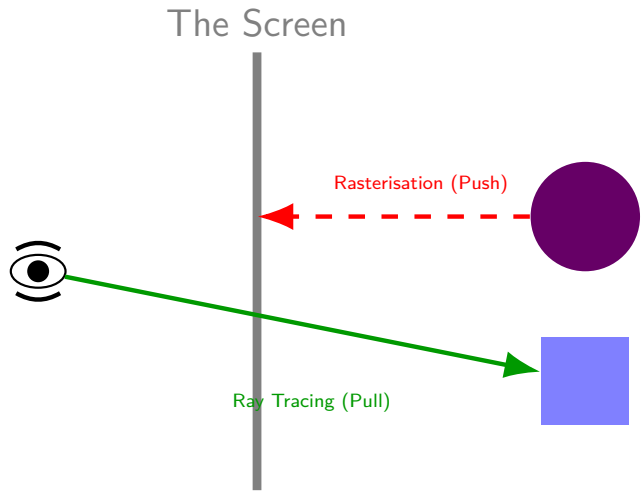


The Plato's Cave of Graphics

The Fundamental Illusion

In philosophy, Plato described prisoners chained in a cave, seeing only shadows of objects cast on a wall. They believed the shadows were the reality.
Computer Graphics is exactly this cave.

- Until now, we have been "Rasterisers". We take a 3D reality and forcefully flatten it onto a 2D wall (the screen).
- But what if, instead of looking at the wall, we turned around and looked at the light itself?
- **Today's Shift:** We stop asking "Where does this triangle land on the screen?" and start asking "How did this photon of light get into my eye?"



Rasterisation vs. Ray Tracing

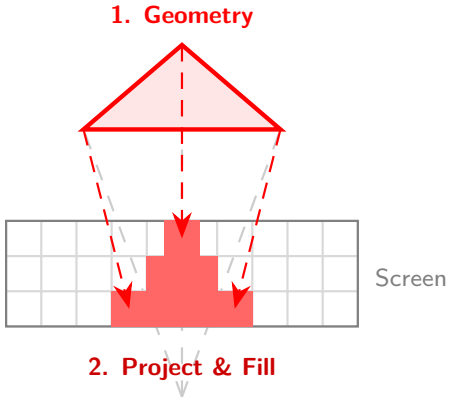
Rasterisation

- **Object-Order**: Loop through Triangles.

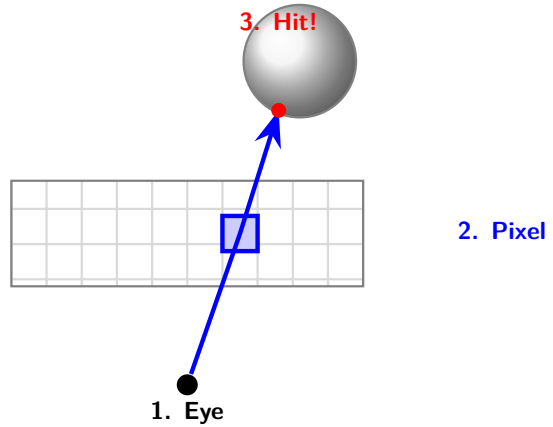
Ray Tracing

- **Image-Order**: Loop through Pixels.

1. Object Order (Forward)



2. Image Order (Backward)



Rasterisation vs. Ray Tracing

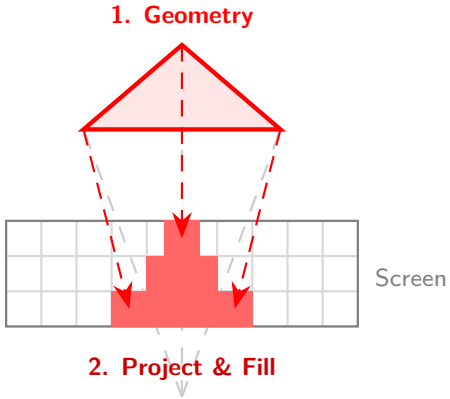
Rasterisation

- **Object-Order**: Loop through Triangles.
- Project 3D geometry onto 2D screen.

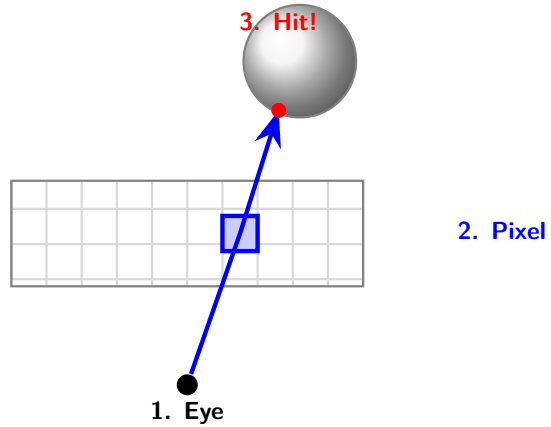
Ray Tracing

- **Image-Order**: Loop through Pixels.
- Cast rays from Camera into the scene.

1. Object Order (Forward)



2. Image Order (Backward)



Rasterisation vs. Ray Tracing

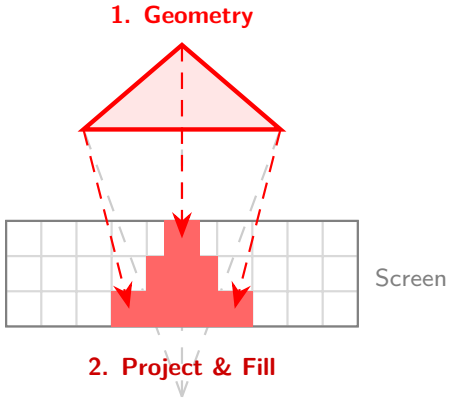
Rasterisation

- **Object-Order**: Loop through Triangles.
- Project 3D geometry onto 2D screen.
- Analogy: "Splattering" paint onto a canvas.

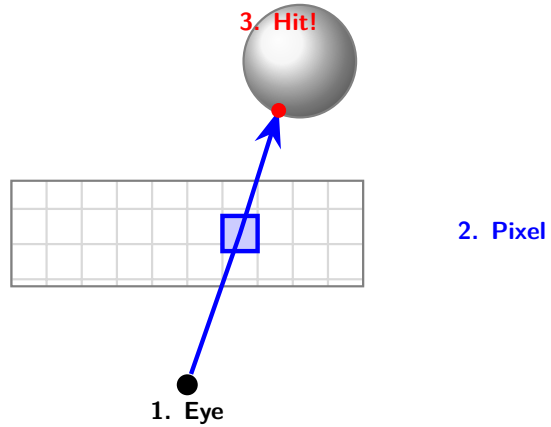
Ray Tracing

- **Image-Order**: Loop through Pixels.
- Cast rays from Camera into the scene.
- Analogy: Tracing light in reverse.

1. Object Order (Forward)



2. Image Order (Backward)



Rasterisation vs. Ray Tracing

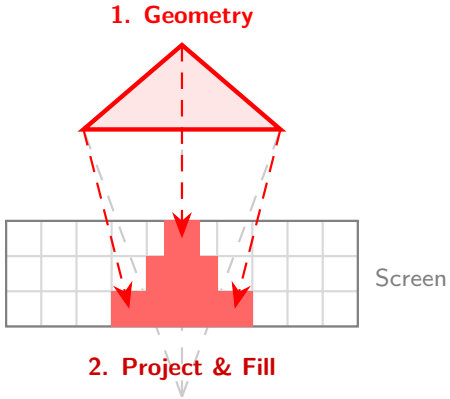
Rasterisation

- **Object-Order**: Loop through Triangles.
- Project 3D geometry onto 2D screen.
- Analogy: "Splattering" paint onto a canvas.
- **Fast:** Complexity $\propto$ Triangles.

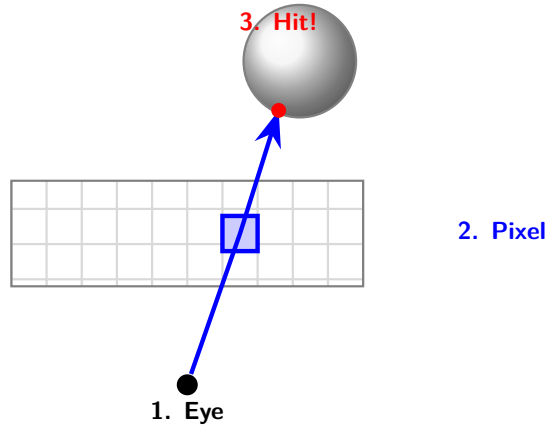
Ray Tracing

- **Image-Order**: Loop through Pixels.
- Cast rays from Camera into the scene.
- Analogy: Tracing light in reverse.
- **Realistic:** Complexity $\propto$ Pixels.

1. Object Order (Forward)



2. Image Order (Backward)



Why Do We Need Ray Tracing?

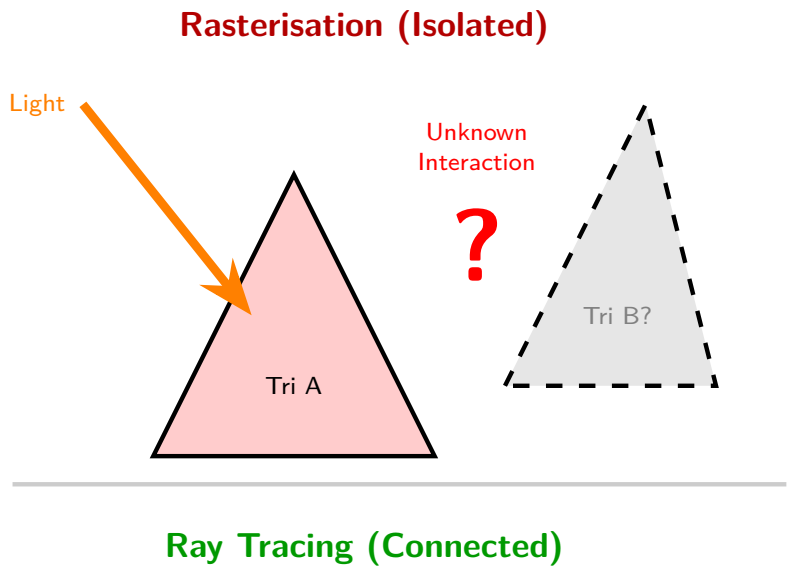
The Problem of Global Illumination

Rasterisation is excellent at determining **visibility** (what you see), but terrible at simulating **light transport** (how light bounces).

The "Local" Trap

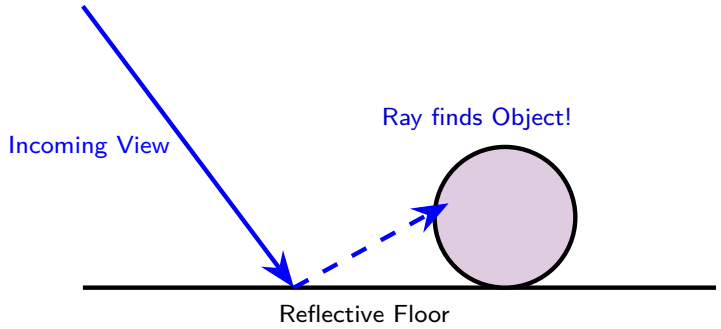
Rasterisation processes one triangle at a time in isolation.

- When drawing Triangle A, the GPU **does not know** Triangle B exists.
- **Result:** It cannot calculate how Triangle B might block light (shadows) or reflect light (mirror) onto Triangle A without complex hacks.



Missing Phenomena:

1. **Reflections:** Mirrors require knowing what is "behind" the ray.
2. **Refraction:** Glass bends light based on thickness.
3. **Soft Shadows:** Area lights are not point sources.



Working Backwards from the Camera

An Image-Order Approach

Reverse Physics

- We trace light paths in **reverse**: from the Eye, through a Pixel, into the Scene.
- This ensures we only calculate lighting for surfaces that are actually visible.
- This is an **image-order** approach, looping through every pixel.

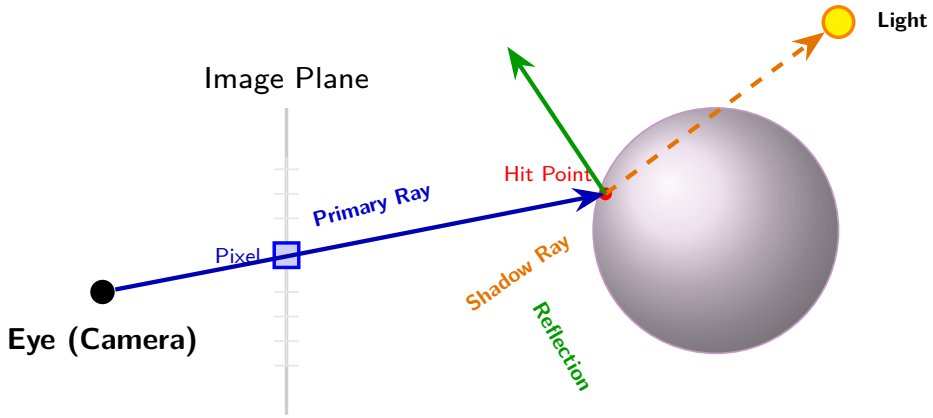


Image-Order Loop:
Calculate contributions for each pixel backward.

Recursive Secondary Rays

- **Shadow Ray:** Checks if the path to the light is blocked.
- **Reflection Ray:** Bounces off surfaces to "see" other objects.

The Result

For every pixel, we resolve a chain of intersections (primary and secondary) to calculate the final color.

The Fundamental Primitive

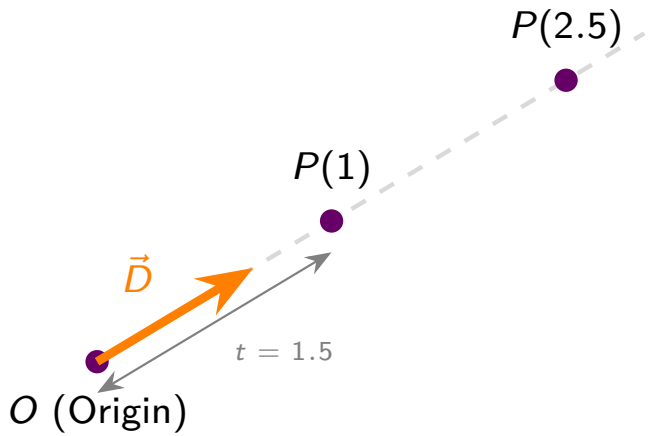
The Mathematical DNA of Light

A ray is a **half-line** defined by its starting point and its heading. This allows us to describe any point in space along the ray's path using a single variable.

Ray Parametric Equation

$$P(t) = O + t\vec{D}$$

- $P(t)$: The resulting **3D position**.
- O: The **Origin** vector (start point).
- $\vec{D}$: The **Normalised Direction** vector ($|\vec{D}| = 1$).
- t: A scalar distance where $t \geq 0$.



t controls how far we travel from the origin.

This definition is the **foundation** for all ray tracing calculations.

Solving for the "Hit" Point

To find if a ray hits a sphere, we look for a point P that satisfies both the **Ray** and the **Sphere** equations simultaneously.

1. Input Equations

Ray Path:

$$P(t) = O + t\vec{D}$$

Sphere Surface:

$$\|P - C\|^2 = r^2$$

2. The Substitution

Insert the Ray into the Sphere:

$$\|(O + t\vec{D}) - C\|^2 = r^2$$

Let $\vec{V} = (O - C)$ to simplify.

The Quadratic Result

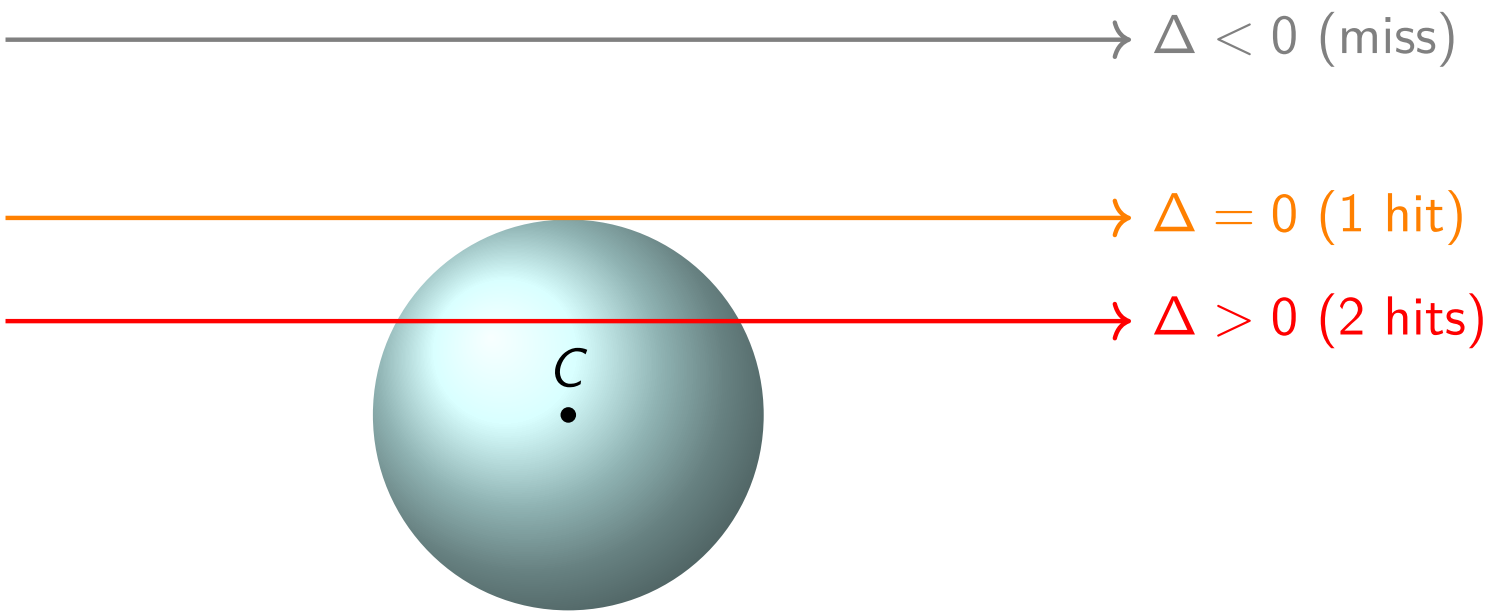
Expanding the dot product gives $at^2 + bt + c = 0$, where:

- $a = \vec{D} \cdot \vec{D} = 1.0$ (because $\vec{D}$ is normalised)
- $b = 2(\vec{D} \cdot \vec{V})$
- $c = (\vec{V} \cdot \vec{V}) - r^2$

What Does the Solution Mean?

The solution to the quadratic equation is determined by the **discriminant**, $\Delta = b^2 - 4ac$.

- If $\Delta < 0$: No real solutions. The ray **misses** the sphere entirely.
- If $\Delta = 0$: One real solution. The ray **grazes** the sphere at a single tangent point.
- If $\Delta > 0$: Two real solutions. The ray passes **through** the sphere, intersecting it at two points.



Implementing the Hit Test

The Logic

We define a ray as $P(t) = Origin + t \cdot Direction$. Substituting this into the sphere equation yields a **quadratic equation**:

$$at^2 + bt + c = 0$$

The Discriminant

The value $b^2 - 4ac$ tells us how many times the ray hits the sphere:

- $< \mathbf{0}$: Misses (0 hits)
- $= \mathbf{0}$: Glances (1 hit)
- $> \mathbf{0}$: Enters & Exits (2 hits)

```
// Ray-Sphere Intersection
function hitSphere(center, radius, ray) {
  // Vector from Center to Origin
  const oc = subtract(ray.origin, center);

  // Coefficients for quadratic eq
  const a = dot(ray.dir, ray.dir);
  const b = 2.0 * dot(oc, ray.dir);
  const c = dot(oc, oc) - radius*radius;

  const discriminant = b*b - 4*a*c;

  if (discriminant < 0) {
    return -1.0; // The Ray missed!
  } else {
    // Return smallest positive t
    // (The hit closest to camera)
    return (-b - Math.sqrt(discriminant))
     / (2.0*a);
  }
}
```

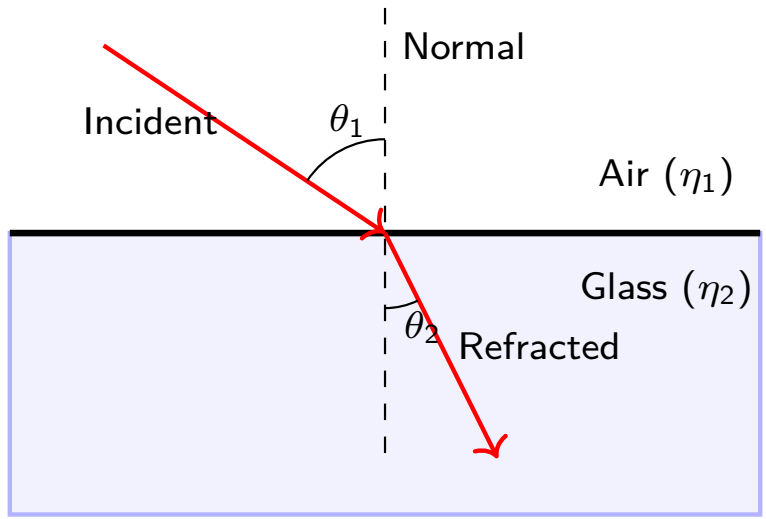
Refraction & Snell's Law

Refraction occurs when a ray passes through a transparent surface, changing direction based on the **Index of Refraction (IOR)** of the materials.

The Mathematics

- **Snell's Law**: $\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$.
- η represents the IOR (e.g., Air ≈ 1.0, Glass ≈ 1.5).
- **Total Internal Reflection**: If the angle is too shallow when exiting a dense medium, the ray reflects instead of refracting.

Graphical Illustration



GLSL Implementation

```
vec3 R = refract(I, N, eta);
```

Note: `eta` is the ratio of IORs ($\frac{n_1}{n_2}$).

- **Implementation Detail**: To prevent the ray from hitting the same surface again, we offset the new ray origin slightly *inside* the object: $P_{new} = P - N \times \epsilon$.
- **Beer's Law**: As light travels through the medium, its color is attenuated based on distance t: $attenuation = e^{-color \times t}$.

Recursive Tracing

To simulate complex light, we don't just stop at the first hit. We act like a pinball machine, bouncing rays recursively.

```
function trace(ray, world, depth) {
  // 1. BASE CASE: Stop if we bounce too many times (prevents infinite loop)
  if (depth <= 0) return backgroundColor;

  // 2. HIT TEST: Check if we hit anything in the world
  const hit = world.intersect(ray);

  if (hit.didHit) {
    // 3. SHADOWS: Cast a ray towards light to see if blocked
    let color = calculateLocalColor(hit);

    // 4. REFLECTION: Recursive Step
    if (hit.material.isReflective) {
      const reflectionRay = createReflectionRay(hit);

      // RECURSE: Call trace() again, but decrease depth!
      const reflectionColor = trace(reflectionRay, world, depth - 1);

      // Blend the reflection with the object's color
      color += reflectionColor * hit.material.reflectivity;
    }
    return color;
  }

  // Ray hit nothing (Background)
  return backgroundColor;
}
```

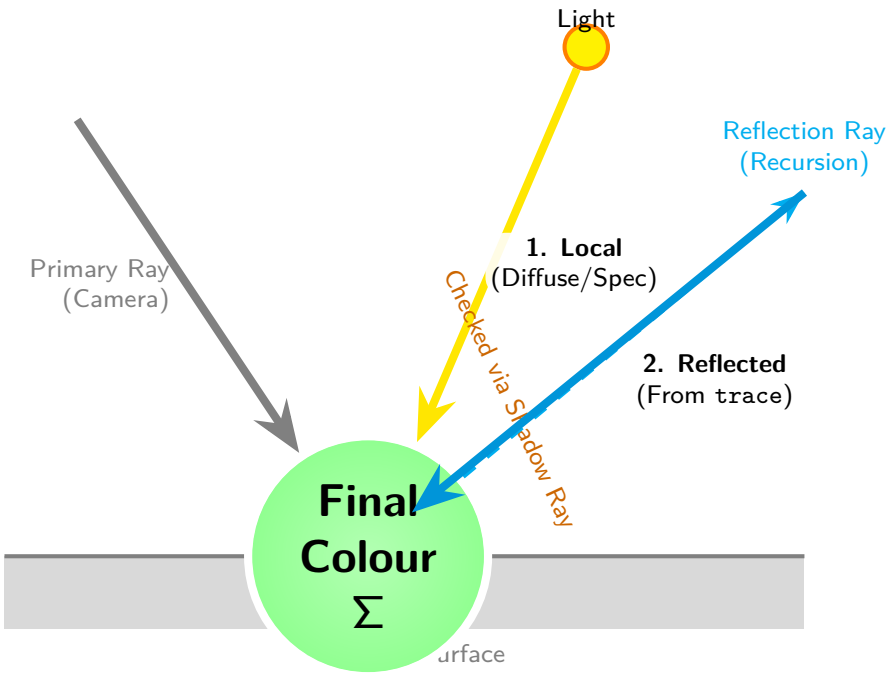
The Sum of Parts

At every intersection point, the ray tracer determines the final **colour** by summing different light contributions. This is the core of the "shading" step in the recursive code.

The Constituents of Colour

The final colour at a hit point is generally a combination of:

1. **Local Illumination:** Direct light from sources (diffuse and specular highlights). This is modulated by **shadow rays**.
2. **Reflected Illumination:** Light arriving from elsewhere in the scene, gathered by the recursive **reflection ray**.



The Simplified Formula

$$C_{final} = (C_{local} \times S_{shadow}) + (C_{reflected} \times K_{reflectivity})$$

- S_{shadow}: 1.0 if lit, 0.0 if in shadow.
- $K_{reflectivity}$: Surface shininess (0.0–1.0).

Calculated Colour: Blinn-Phong

While ray tracing handles global effects (shadows), we need a specific formula to calculate the **local colour** at a hit point. The most common approach is the **Blinn-Phong Model**.

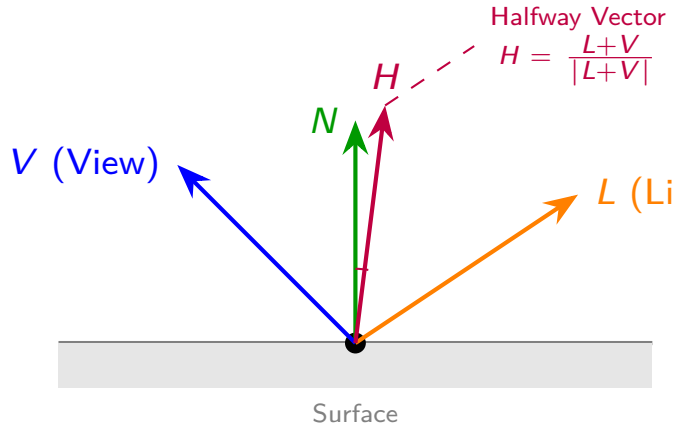
Why Blinn-Phong?

It is an **optimised** variation of the classic Phong model. Instead of calculating a costly reflection vector, it uses a "Halfway Vector" (H) to estimate shininess.

The Three Components

The final local colour is a sum of three parts:

1. **Ambient:** Constant base colour.
2. **Diffuse:** Matte colour. Depends on the angle between Light (L) and Normal (N).
3. **Specular:** The highlight. Depends on the angle between Normal (N) and the **Halfway Vector (H)**.



The Formula

$$I = \underbrace{k_a}_{\text{Amb}} + \underbrace{k_d(L \cdot N)}_{\text{Diff}} + \underbrace{k_s(N \cdot H)^{\alpha}}_{\text{Spec}}$$

The Ray Tracing Pipeline: Primary Rays & Intersection

Ray tracing is an **image-order** process. For every pixel, we determine visibility and distance through a "hit test" against the scene geometry.

Component	Analytical Intersection (Geometric)	Signed Distance Functions (SDF)
View Ray Emission	Ray: $P(t) = O + t\vec{D}$	Same; starts at camera origin O in direction $\vec{D}$
Distance Check	**Quadratic Formula**: Solve $at^2 + bt + c = 0$ for spheres.	**Ray Marching**: Step along ray by distance to nearest surface.
Math/Logic	Discriminant $\Delta = b^2 - 4ac$. Smallest $t > 0$ is the hit.	$t = t + \text{dist}(p)$. Stop when dist $<$ threshold.
Organisation	**Hierarchical Clustering (BVH)**: Test ray against bounding boxes first.	**CSG Operations**: Combine shapes using `min(d1, d2)` or `max(d1, -d2)`.
Pros/Cons	**Exact** and fast for simple shapes. Complex for fractals/organic noise.	**Flexible** for complex blending; requires many "steps" (performance heavy).

- **Sampling Awareness**: Use **Anti-Aliasing (AA)** by shooting multiple jittered rays (2×2 grid) per pixel and averaging the result to smooth edges.

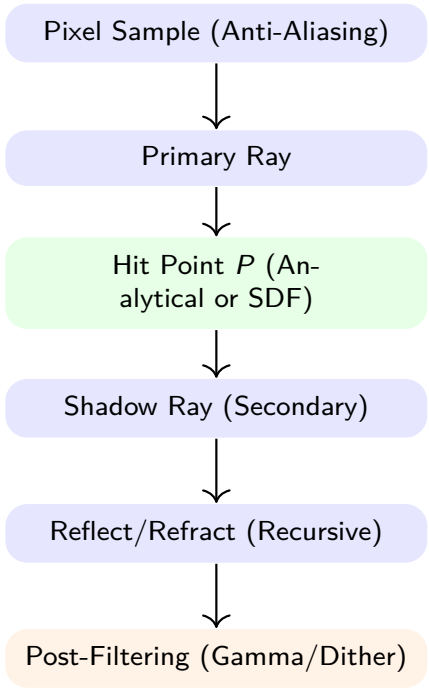
Recursive Shading & Post-Filter

Once a hit is found, the final color is an **accumulation** of direct light and recursive secondary rays.

Color Contribution Calculation

- **Shadow Rays**: Cast ray from hit point P to Light L. If blocked, $Shadow = 0$.
- **Reflection**: $R = \text{reflect}(D, N)$. Recursive call to `trace()` with depth limit.
- **Refraction**: **Snell's Law**. Uses Index of Refraction (IOR) to bend rays through glass.
- **Accumulation**: $C_{final} = C_{local} + (Reflectivity \times C_{refl})$.

Workflow Summary



Attention: Post-Filtering

Dithering is applied in the final pass to turn 8-bit color banding into imperceptible noise. **Gamma Correction** ($C^{1/2.2}$) is required for realistic display on monitors.

The Hollywood Lie

Why Rendering is Not Enough

Imagine you have just filmed the perfect scene for a movie. The actors were great, the lighting was perfect.

The Hollywood Lie

Why Rendering is Not Enough

Imagine you have just filmed the perfect scene for a movie. The actors were great, the lighting was perfect.
Is the movie ready for the cinema? No.

The Hollywood Lie

Why Rendering is Not Enough

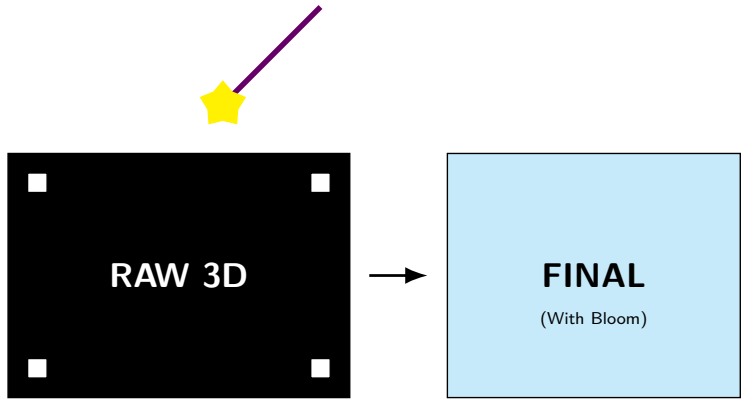
Imagine you have just filmed the perfect scene for a movie. The actors were great, the lighting was perfect.
Is the movie ready for the cinema? No.

Raw footage is boring. It needs color grading, lens flares, blur, and grain.

In graphics, we have the same problem.

- Rendering your 3D geometry is just "filming on set."
- **Post-Processing** is the "editing room."

The Lie: Most of the "atmosphere" in games (fog, bloom, motion blur) is not 3D at all. It is 2D Photoshop applied *after* the rendering is done.



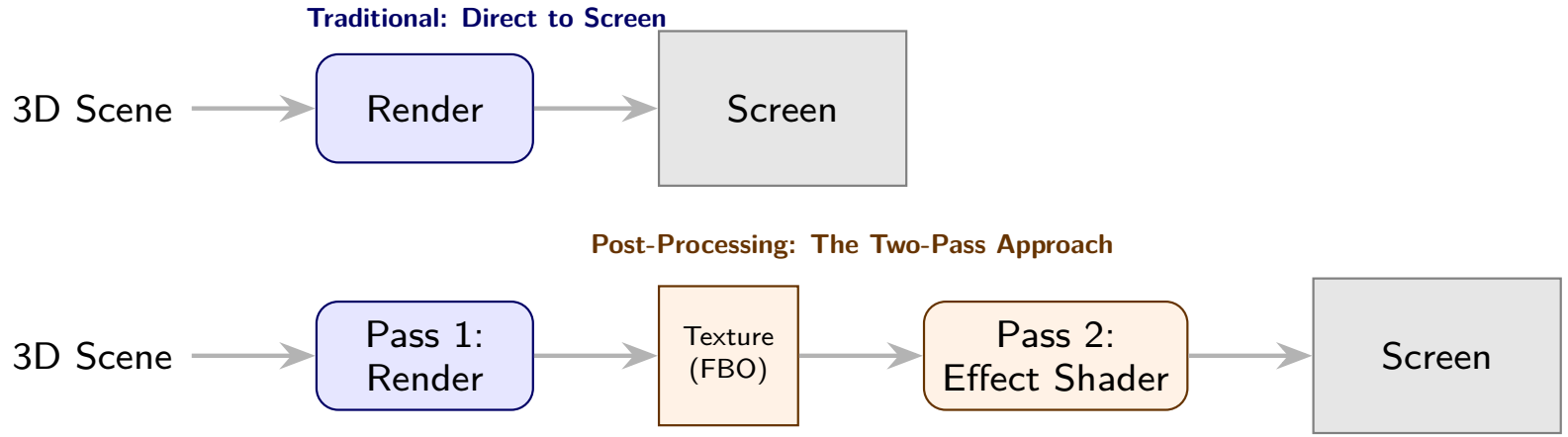
Why Render to Something Other Than the Screen?

So far, our render pipeline's only destination has been the default framebuffer (i.e., the screen).

What if we want to apply an effect to the entire rendered image at once?

- Applying a blur to simulate motion or depth of field.
- Adjusting brightness, contrast, or color balance.
- Creating stylistic filters like film grain, vignettes, or greyscale.

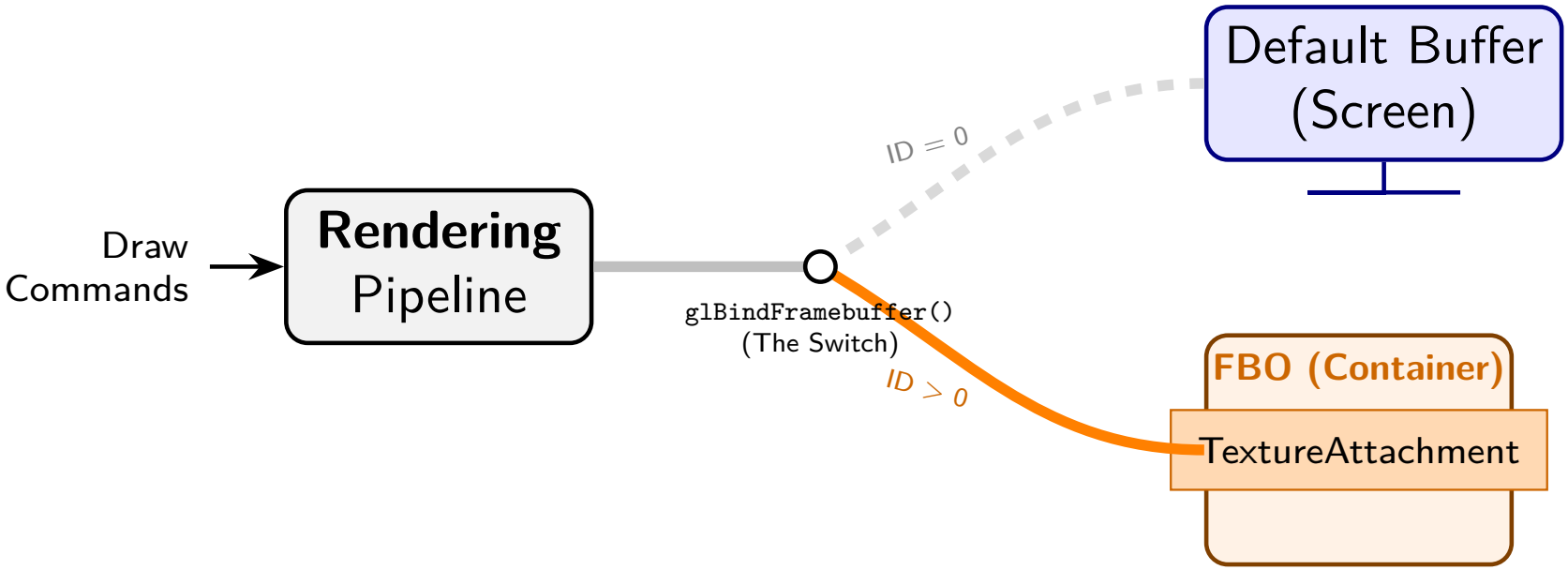
These are called **post-processing** effects. To achieve them, we need to render our 3D scene to a texture first, instead of directly to the screen.



Your Off-Screen Canvas

A **Framebuffer Object (FBO)** is a mechanism that allows us to redirect the rendering pipeline to an off-screen destination.

- An FBO is a container for a collection of "attachments".
- The most important attachment is a **texture**, which will act as our new color buffer.
- When we "bind" a custom FBO, all subsequent drawing commands will write their results into this texture instead of the screen.

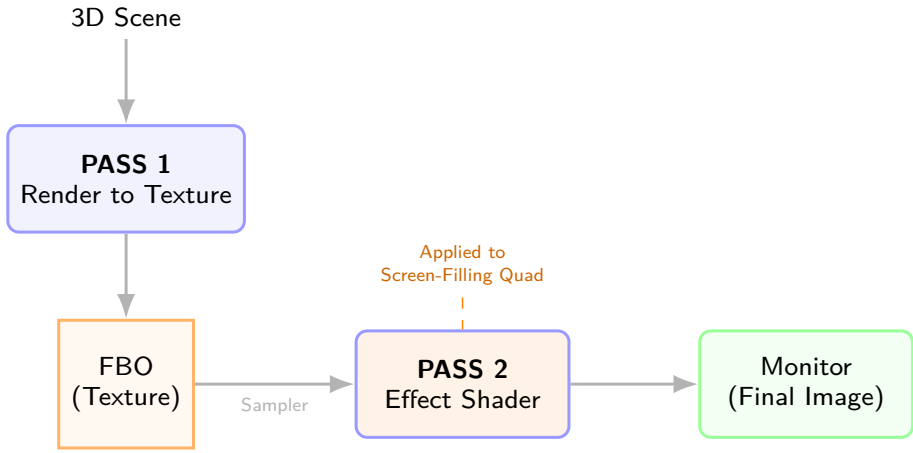


Post-Processing Strategy

Capturing Reality to Apply an Illusion

Strategy: Render twice, see once.

1. **Pass 1 (Capture):** Redirect the GPU to an **FBO**. The 3D world is "baked" into a 2D texture.
2. **Pass 2 (Filter):** Draw a 2D **Screen Quad**. Project the Pass 1 texture onto it using an "Effect Shader".

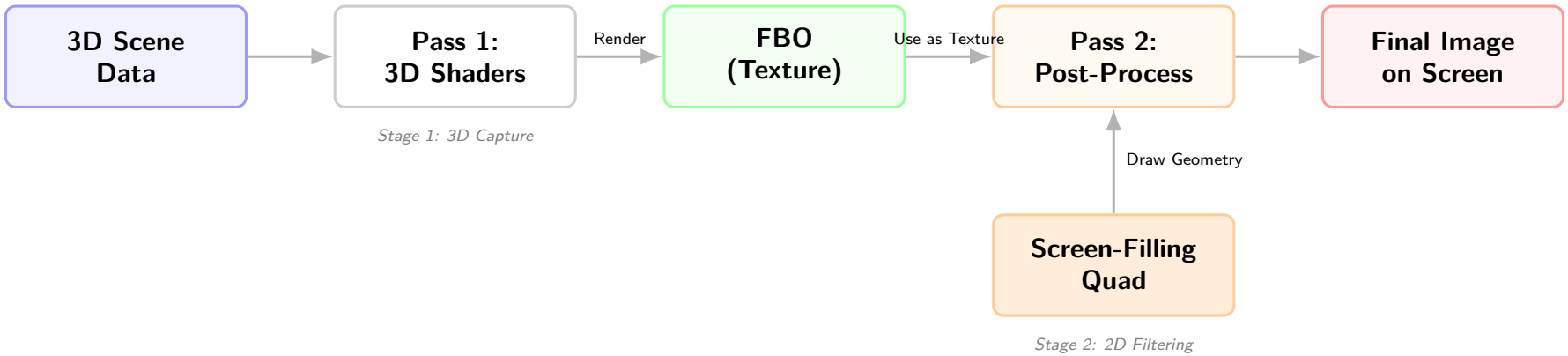


Why do this?

Effects like **Bloom, Blur, and Fog** are often too expensive for 3D triangles, but very easy to "fake" on a 2D image.

Visualising the Post-Processing Pipeline

From 3D Geometry to 2D Pixel Manipulation



- **Efficiency via Decoupling:** Once the scene is "baked" into the FBO texture, the cost of post-processing is independent of the number of triangles in your 3D world.
  - *Generalisation:* This allows for "chained" effects (e.g., Bloom → Color Grading → UI Overlay) by treating the output of one pass as the input for the next without re-rendering the scene.
- **The Quad as Viewport Mapping:** The Screen-Filling Quad is not just a surface; it acts as a 2D coordinate system for the entire screen.
  - *Generalisation:* By using the quad's texture coordinates ($v_{texCoord}$), you can create "Screen-Space" effects like Vignetting or Depth-of-Field, where the math depends on the pixel's (x, y) position relative to the screen center.

The "Filter" Mental Model

Think of the **FBO Texture** as raw film footage and the **Screen Quad** as the projector screen. The **Post-Process Shader** is the digital lens that can change the mood, focus, or clarity of the image before it hits the viewer's eye.

Greyscale Fragment Shader

Durham University

This is the shader used in **Pass 2**. It takes the scene texture as input and modifies it.

```
// Post-processing Fragment Shader
precision mediump float;
// The texture from Pass 1
uniform sampler2D u_sceneTexture;

// The texture coordinate from the quad's vertices
varying vec2 v_texCoord;

void main() {
  // Sample the color from the scene texture
  vec4 sceneColor = texture2D(u_sceneTexture, v_texCoord);

  // Calculate greyscale using luminance weights
  float luminance = 0.299*sceneColor.r + 0.587*sceneColor.g + 0.114*
     sceneColor.b;

  // Set the final pixel color
  gl_FragColor = vec4(luminance, luminance, luminance, 1.0);
}
```

Listing 1: A simple GLSL shader to convert a color image to greyscale.

Box Blur Fragment Shader

A blur is achieved by sampling neighboring pixels and averaging their colors.

```
uniform sampler2D u_sceneTexture;
uniform vec2 u_texelSize;
// Size of one pixel: (1.0/width, 1.0/height)
varying vec2 v_texCoord;

void main() {
  vec4 total = vec4(0.0);

  // Iterate over a 3x3 grid of pixels
  for (int x = -1; x <= 1; x++) {
    for (int y = -1; y <= 1; y++) {
      vec2 offset = vec2(float(x), float(y)) * u_texelSize;
      total += texture2D(u_sceneTexture, v_texCoord + offset);
    }
  }

  // Average the 9 samples
  gl_FragColor = total / 9.0;
}
```

Listing 2: A 3x3 box blur averages 9 pixel samples.

We have mastered the core pipeline. Now, we add the "upgrades" that transform a collection of triangles into a living, breathing world.

Dynamic Geometry

Shadow Mapping: Multi-pass logic to determine which pixels are "hidden" from light sources.

Skinned Animation: Deforming meshes smoothly using a weighted virtual skeleton.

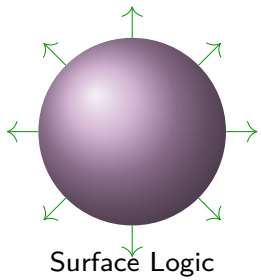
Surface Realism

Normal Mapping: Using textures to "fake" millions of polygons of detail on a flat surface.

PBR: A lighting model based on physical properties like *Roughness* and *Metallic*.

Complex Systems

Particle Systems: Managing mass entities (fire, smoke) using emitter logic and billboards.



These techniques represent the shift from "How do I draw?" to "How do I simulate?"

Real-Time Shadows in Rasterisation

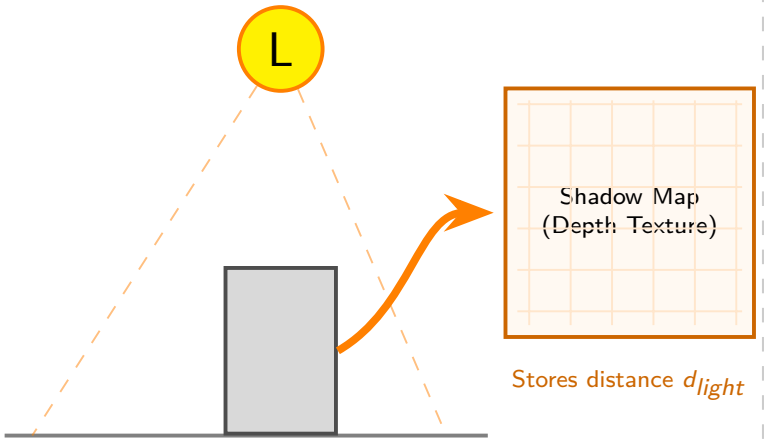
While ray tracing produces perfect shadows, it is too slow for many real-time applications. **Shadow Mapping** is the standard Rasterisation alternative.

The Core Idea

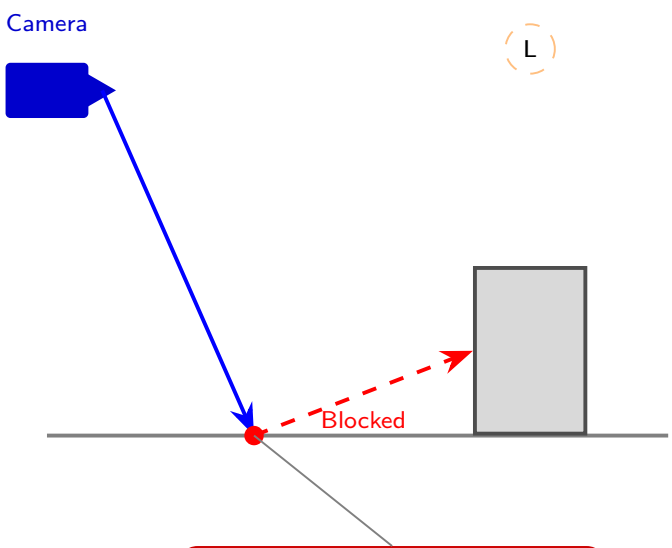
If a point in the scene cannot be "seen" by the light source, it must be in shadow. We determine this by rendering the scene from the light's perspective first.

This is another multi-pass technique that uses an FBO.

Pass 1: Render Depth



Pass 2: Render Scene



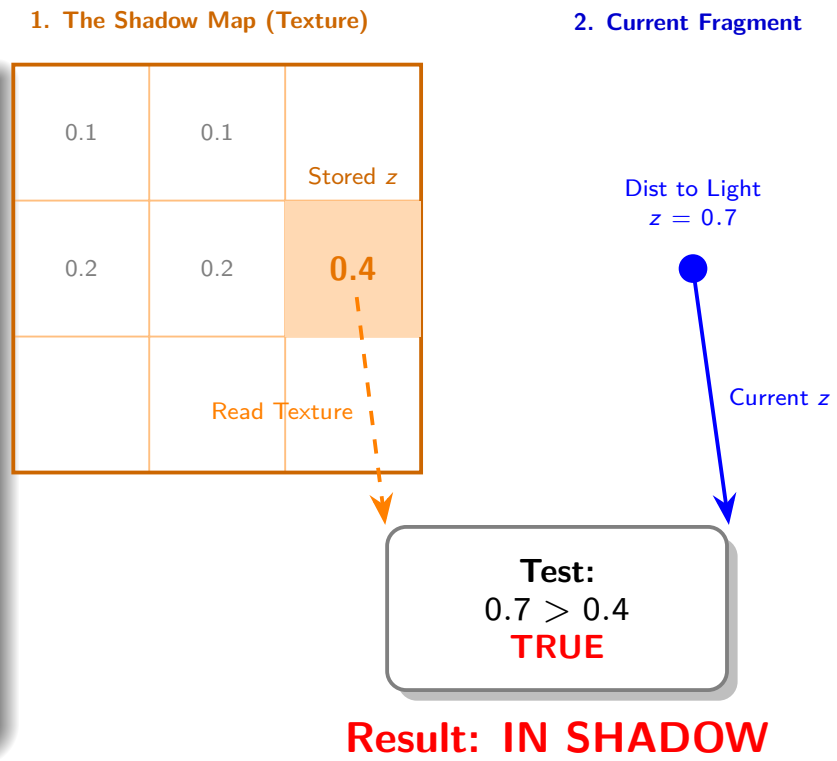
The Depth Comparison Test

Once we have the Shadow Map (Pass 1), how do we use it in the final render (Pass 2)? Inside the fragment shader, we perform a coordinate transformation and a comparison.

The Shader Logic

For every fragment on screen:

1. Calculate the fragment's position from the **Light's** point of view.
2. Look up the stored depth value in the Shadow Map texture at those (u, v) coordinates.
3. **Compare:**
   - If **Current Depth** $>$ **Stored Depth**: The fragment is behind something. $\rightarrow$ **SHADOW**
   - Otherwise: $\rightarrow$ **LIT**



Common Artifact: "Shadow Acne"

Due to limited resolution, shadows often flicker on surfaces. We fix this by adding a small "bias" value to the comparison.

Animating Characters with Bones

Skinned Animation is the standard method for deforming character meshes smoothly (e.g., an elbow bending).

1. The Skeleton

A hierarchy of unseen **Joints** (Bones) is built inside the mesh.

- Shoulder $\rightarrow$ Elbow $\rightarrow$ Wrist

2. The Skin (Mesh)

Each vertex is assigned **Weights** describing how much it follows each bone.

- Usually 1–4 bones per vertex.

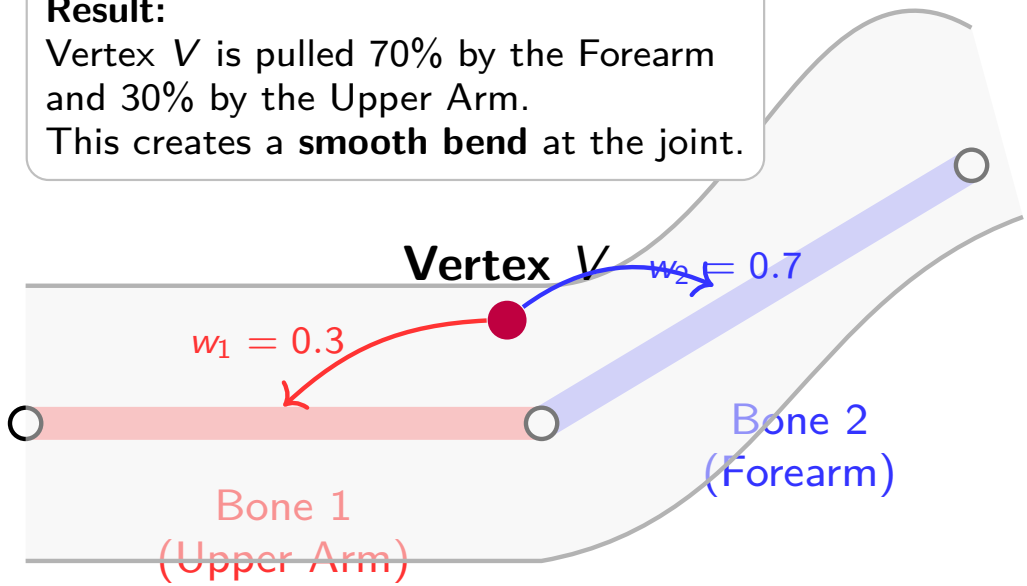
The Vertex Shader Calculation

$$P_{final} = \sum_{i=1}^{n} (P \cdot M_{bone,i} \cdot w_i)$$

The final position is a **weighted average** of where each bone wants the vertex to be.

Visualising Vertex Influences

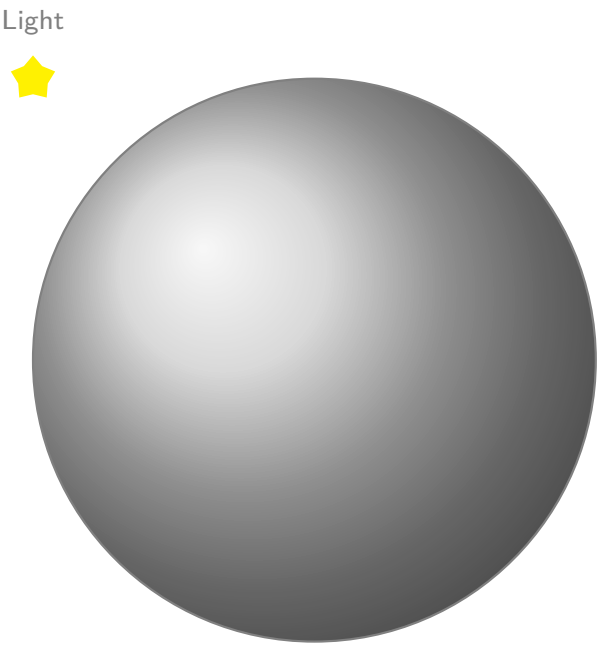
Result:
Vertex V is pulled 70% by the Forearm and 30% by the Upper Arm.
This creates a **smooth bend** at the joint.



Visualising the Effect

Without Normal Map

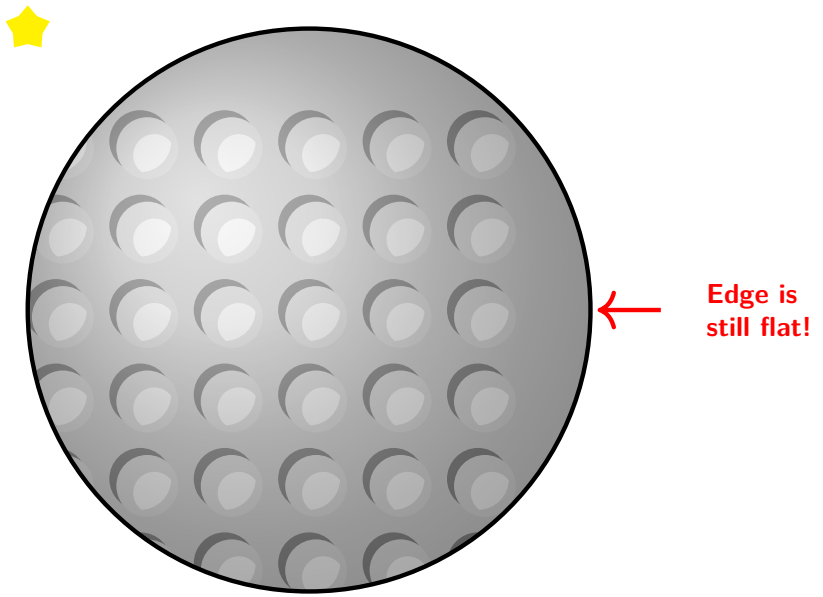
- A simple, low-polygon sphere.
- Surface appears perfectly smooth.
- Lighting relies solely on interpolated vertex normals.



Flat Geometry
Smooth Shading

With Normal Map

- Same low-poly geometry.
- Lighting sampled from texture.
- **Key:** Surface looks bumpy, but the outline is round.



Fake Depth
(Dimples)

A More Realistic Shading Model

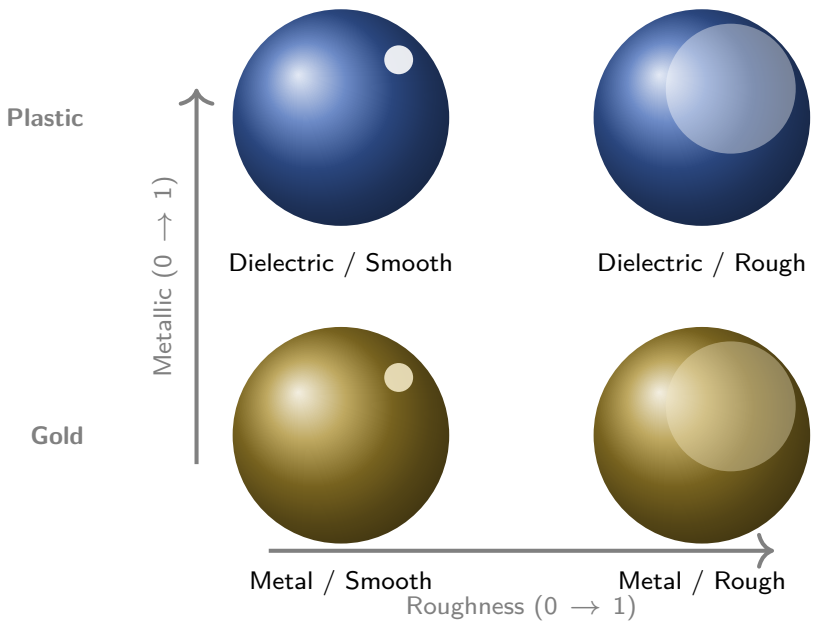
PBR models the interaction of light and materials using physically plausible properties rather than abstract "shininess."

The Workflow

The standard **Metallic/Roughness** setup uses three key maps:

- **Albedo**: Base colour (RGB).
- **Metallic**: 0.0 (Insulator) to 1.0 (Conductor).
- **Roughness**: 0.0 (Mirror) to 1.0 (Chalk).

Result: Consistent look in all lighting.



The Mathematics of Microfacets

Energy Conservation & Surface Physics

The BRDF Model

PBR uses a **BRDF** to calculate how light scatters off micro-scale surface details.

- **Microfacet Theory:** We treat every surface as a collection of millions of tiny, microscopic mirrors.
- **Statistical Shading:** Roughness determines the "chaos" of these mirrorsscattered mirrors create blurry reflections.

Smooth: Aligned Facets

Rough: Chaotic Facets

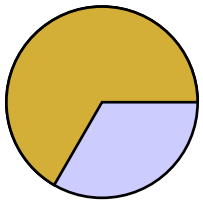
Energy Conservation

Light cannot be created, only reflected or absorbed. PBR enforces a strict physical limit:

$$\text{Diffuse} + \text{Specular} \leq 1.0$$

If a metal reflects 90% of light, it has only 10% left for diffuse color.

Energy Balance



High Specular = Low Diffuse

Why PBR Needs an FBO

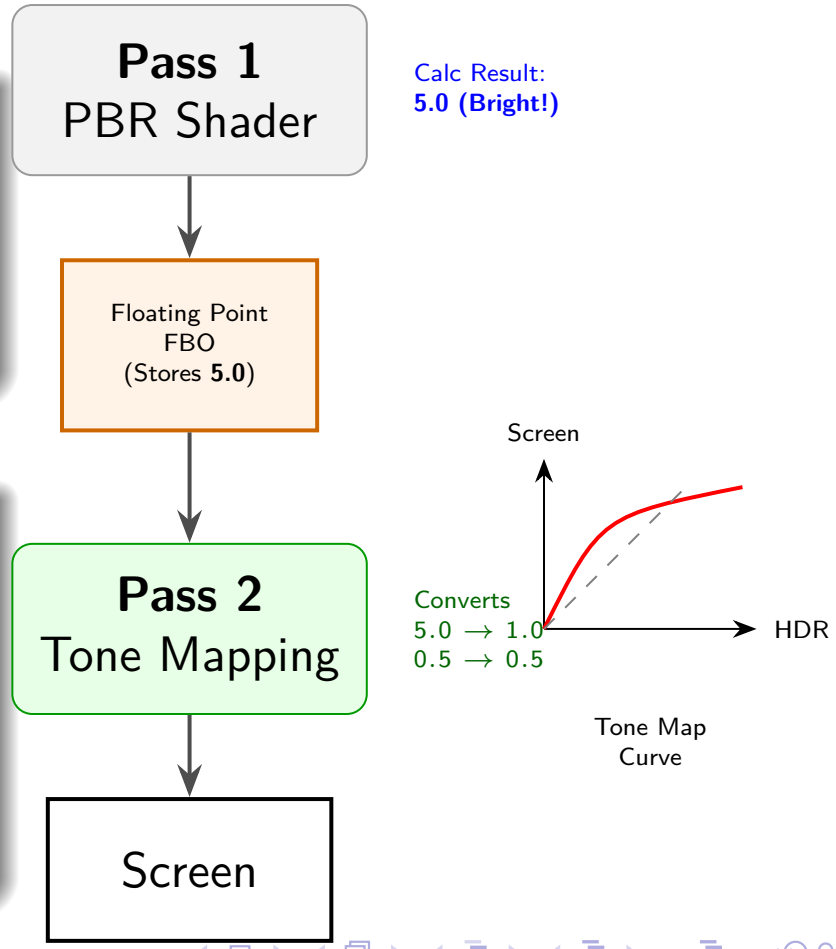
Implementing PBR is not just about the shader math. It changes the pipeline structure. PBR lighting calculations often produce light values much greater than **1.0**.

The Dynamic Range Problem

Monitors can only display colours between 0.0 and 1.0. If a PBR reflection calculates a brightness of 5.0, rendering directly to the screen "clamps" it to 1.0, losing all detail.

The 2-Pass Solution (HDR)

1. **Pass 1 (FBO):** Render the PBR scene into a **Floating Point Texture**. This special FBO can store values > 1.0 without clamping.
2. **Pass 2 (Post-Process):** Read the FBO and apply **Tone Mapping**. This mathematically "squashes" the bright values down to the $0.0 \rightarrow 1.0$ range, mimicking how a human eye adjusts to bright light.



Simulating "Fuzzy" Phenomena

Particle Systems are used for simulating chaotic or fluid phenomena that are difficult to model with traditional geometry, such as fire, smoke, sparks, rain, and explosions.

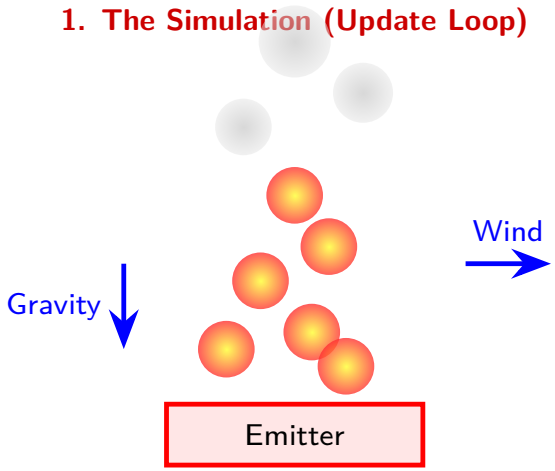
The Concept

A system manages a large number of simple objects, called **particles**. Each is an independent entity with attributes like position, velocity, colour, size, and lifetime.

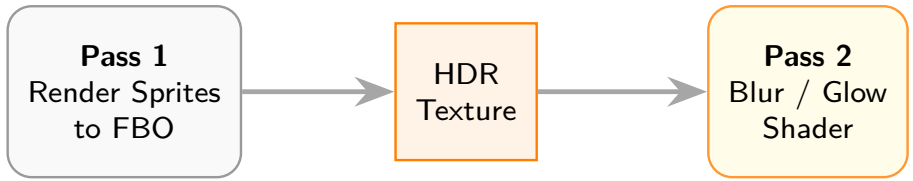
The Process

An **emitter** generates particles. Every frame, physics (gravity, wind) update their state. They are rendered as camera-facing 2D sprites ("billboards").

1. The Simulation (Update Loop)



2. The Render Pipeline (Why FBO?)



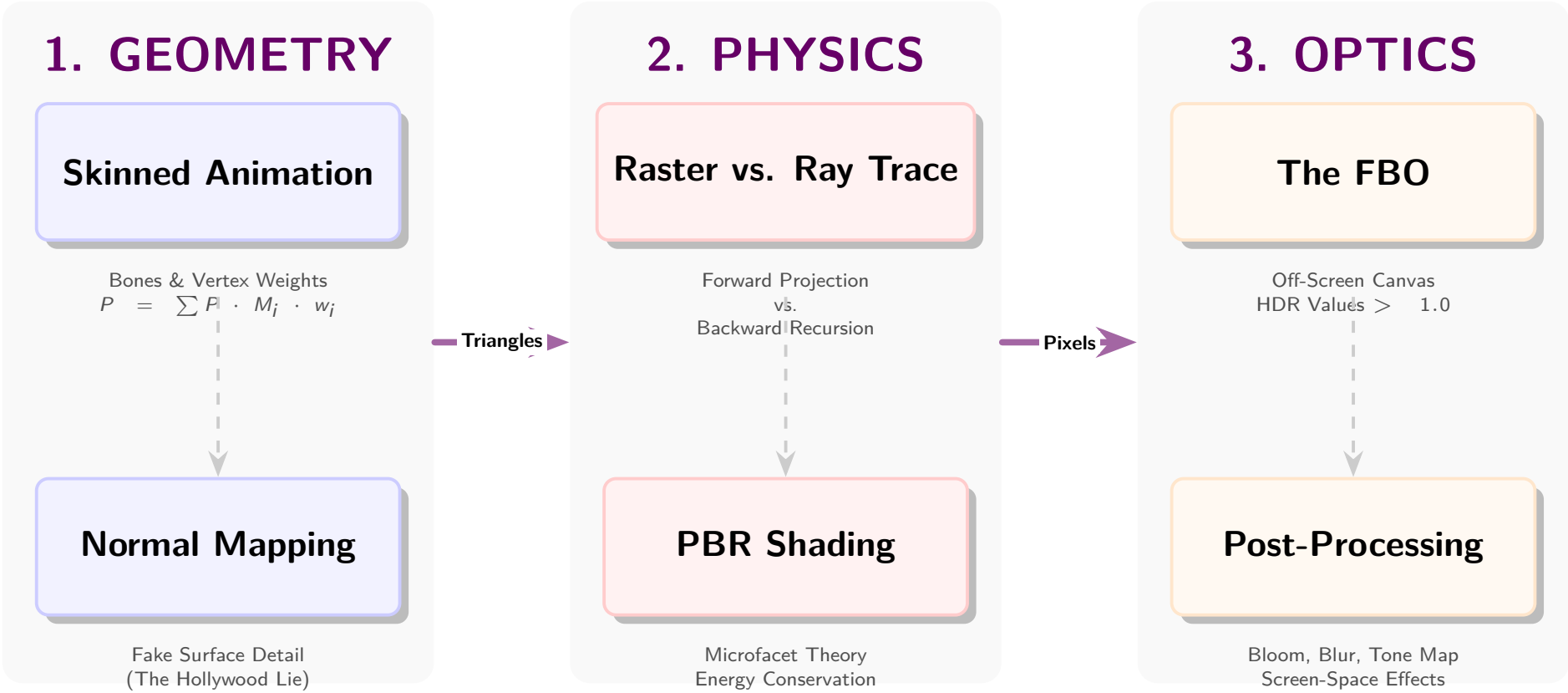
Particles look "flat" on their own.
We use **Pass 2** to add Glow/Bloom
so fire looks bright!

Key Takeaways

- **Ray Tracing**: An image-order technique that simulates light transport for ultimate realism, at a high computational cost.
- **Post-Processing**: A powerful method using **FBOs** to apply full-screen effects by rendering the scene to a texture first.
- **Shadow Mapping**: The standard real-time Rasterisation technique for shadows, which works by rendering a depth map from the light's perspective.
- **Skinned Animation**: Efficiently animates characters by deforming a mesh based on the movement of an underlying skeleton.
- **Normal Mapping**: Creates the illusion of high-resolution geometry on low-poly models by manipulating normals during lighting calculations.
- **PBR**: Provides more realistic and consistent materials by using physically-based properties like roughness and metalness.

From Polygons to Photorealism

Consolidating the Modern Rendering Pipeline



The Grand Takeaway:

- **Deception:** We use Normal Maps and Post-Processing (FBOs) to fake detail we can't afford to render.
- **Simulation:** We use PBR and Ray Tracing to mimic the actual physics of light.
- **The Pipeline:** It is no longer just "draw triangles." It is a multi-stage simulation of a camera.

Questions?

Study Guide: Mastering Advanced Rendering

A Roadmap from Theory to Implementation

To master this lecture, approach the topics in this order:

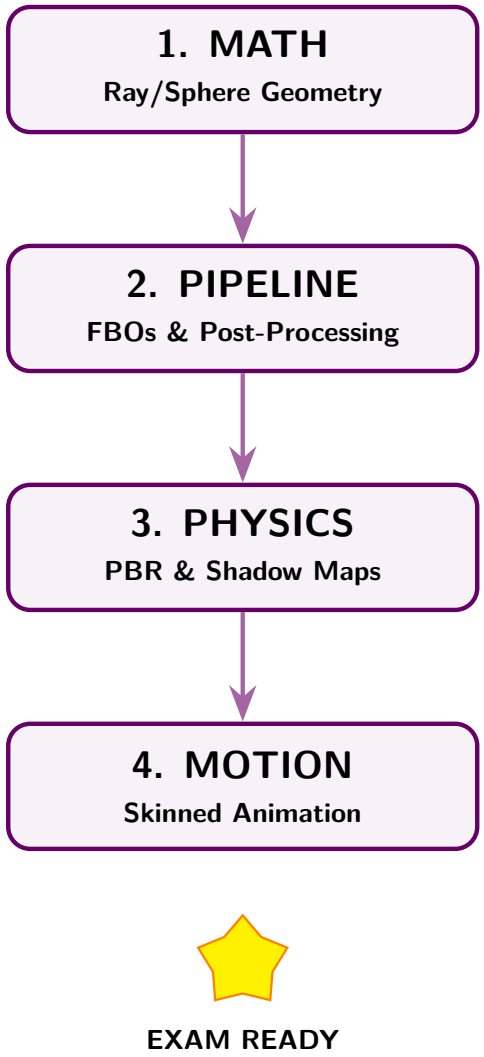
Phase 1: Foundations *Understand the "Why".* Compare **Ray Tracing** vs. **Rasterisation**. Focus on the math of the Ray-Sphere intersection ($at^2 + bt + c = 0$).

Phase 2: Infrastructure *Master the "How".* Learn how the **FBO** acts as an off-screen canvas. Study the **Two-Pass** logic: Render to texture $\rightarrow$ Apply Shader $\rightarrow$ Screen.

Phase 3: Realism *Apply the "Upgrades".* Connect **PBR** (Energy Conservation) with **Normal Mapping** (Fake detail) and **Skinned Animation** (Vertex weighting).

Pro-Tip for Revision

Don't just read the codedraw the rays! For every technique (**Shadow Mapping**, **PBR**, **Reflections**), ask: *"Which way is the photon moving, and is the calculation happening in 3D space or 2D screen space?"*



Appendix: Implementing Shadows

The "Opt-In" Pipeline

Three.js automates the complex FBO/Depth Map creation , but because shadows are expensive, they are disabled by default. You must explicitly enable them at three levels: the **Renderer**, the **Light**, and the **Mesh**.

Three.js Code Example

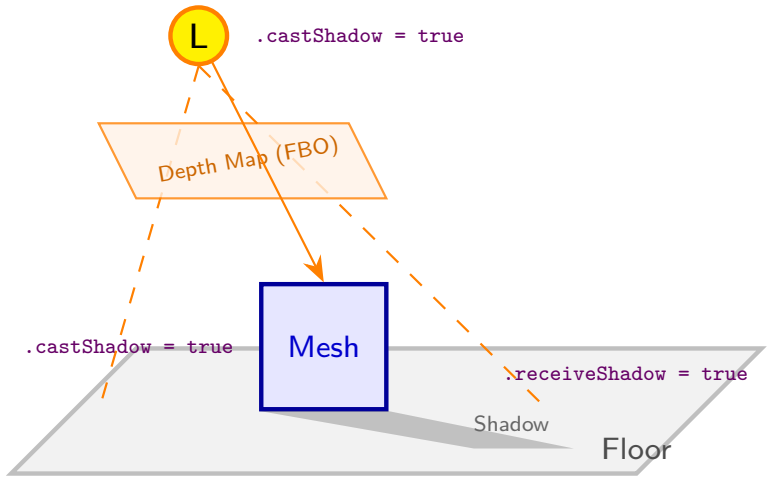
```
// 1. ENABLE ON RENDERER (The Pipeline)
const renderer = new THREE.WebGLRenderer();
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;

// 2. CONFIGURE LIGHT (The Source)
const light = new THREE.DirectionalLight(0xffffff, 1);
light.position.set(5, 10, 5);
light.castShadow = true;

// Optional: Increase Shadow Map Resolution (FBO Size)
light.shadow.mapSize.width = 1024;
light.shadow.mapSize.height = 1024;
scene.add(light);

// 3. CONFIGURE MESHES (The Interaction)
const cube = new THREE.Mesh(geo, mat);
cube.castShadow = true;      // Creates shadow
cube.receiveShadow = false; // (Optional optimization)
scene.add(cube);

const floor = new THREE.Mesh(planeGeo, mat);
floor.receiveShadow = true; // Surface shadows land on
scene.add(floor);
```



Key Note

The `shadow.mapSize` determines the quality. Low values (e.g., 256) cause "blocky" shadows; high values (e.g., 2048) consume more VRAM.

Appendix: Implementing Normal Mapping

Loading and Applying Normal Maps

To implement Normal Mapping , we do not change the geometry. Instead, we load a special texture where RGB values represent XYZ normal vectors and assign it to a compatible material (e.g., `MeshStandardMaterial`).

Three.js Code Example

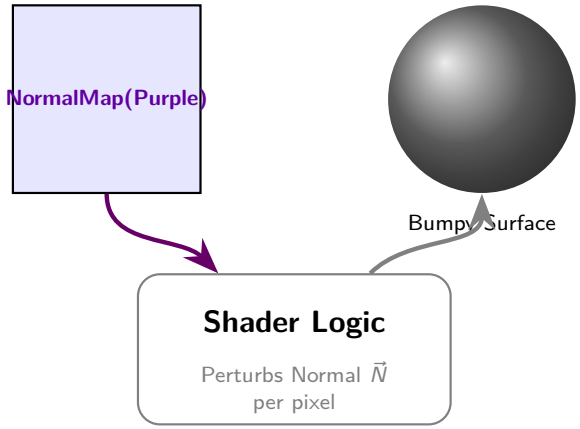
```
// 1. LOAD TEXTURES
const loader = new THREE.TextureLoader();

// Load the base color (Albedo)
const colorMap = loader.load('textures/brick_diffuse.jpg');

// Load the Normal Map (RGB = XYZ data)
const normalMap = loader.load('textures/brick_normal.jpg');

// 2. CREATE MATERIAL
const material = new THREE.MeshStandardMaterial({
    map: colorMap,              // The visual color
    normalMap: normalMap,       // The fake depth data
    normalScale: new THREE.Vector2(1, 1) // Intensity (x, y
       )
});

// 3. CREATE MESH
// Note: Geometry must have UV coordinates!
// (Standard primitives like BoxGeometry have UVs by
      default)
const geometry = new THREE.BoxGeometry(1, 1, 1);
const mesh = new THREE.Mesh(geometry, material);
scene.add(mesh);
```



Requirement: Your geometry needs **UV** coordinates so the engine knows how to wrap the flat texture around the 3D shape.

Appendix: PBR Implementation

Using MeshStandardMaterial

Three.js provides the `MeshStandardMaterial` as its primary PBR shader. It unifies lighting calculations using just two main sliders: **Roughness** and **Metalness**.

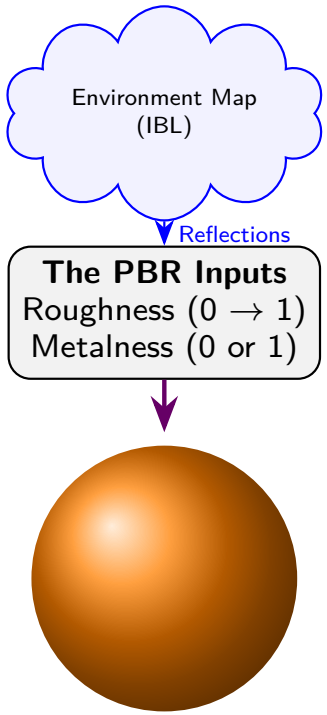
Three.js Code Example

```
// 1. SETUP ENVIRONMENT MAP (Essential for PBR)
// PBR needs a background to reflect!
new RGBELoader().load('env_map.hdr', function(texture) {
    texture.mapping = THREE.
      EquirectangularReflectionMapping;
    scene.environment = texture;
});

// 2. DEFINE MATERIAL
const pbrMat = new THREE.MeshStandardMaterial({
    color: 0xffaa00,       // Albedo (Base Color)
    roughness: 0.2,        // 0.0 (Mirror) to 1.0 (Matte)
    metalness: 1.0,        // 1.0 (Metal) or 0.0 (Dielectric
      )

    // Optional: Load texture maps for detail
    roughnessMap: loader.load('textures/rough.jpg'),
    metalnessMap: loader.load('textures/metal.jpg'),
    normalMap:    loader.load('textures/normal.jpg')
});

// 3. CREATE MESH
const sphere = new THREE.Mesh( geometry, pbrMat );
scene.add(sphere);
```



Gold Sphere

Why an Environment Map?

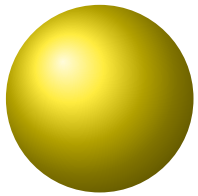
PBR materials look fake without an environment map because shiny metals need something to reflect. In Three.js, we assign this via `scene.environment`.

Appendix: Material Cheatsheet

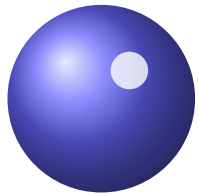
Real-World Materials vs. Three.js Settings

Different real-world materials can be simulated by changing only the `roughness` and `metalness` values .

Material Type	Metalness	Roughness	Visual Description
Mirror / Chrome	1.0	0.0	Perfect reflection, no diffuse color.
Brushed Metal	1.0	0.3 – 0.5	blurry reflections, still looks metallic.
Gold / Copper	1.0	0.1 – 0.2	Tinted reflections (Base Color tints the reflection).
Shiny Plastic	0.0	0.0 – 0.2	Sharp white highlights on colored base.
Matte Plastic/Rubber	0.0	0.8 – 1.0	No distinct highlights, flat look.
Ceramic / Tile	0.0	0.1 – 0.3	Smooth, glossy dielectric surface.
Stone / Concrete	0.0	0.9 – 1.0	Very rough, almost no specular reflection.



Metal = 1.0



Metal = 0.0

Appendix: Post-Processing Example

Adding "Glow" with EffectComposer

In Three.js, we do not use the standard 'renderer.render()'. Instead, we use an **EffectComposer** to chain multiple passes together (Render → Effect → Screen).

Three.js Code Example

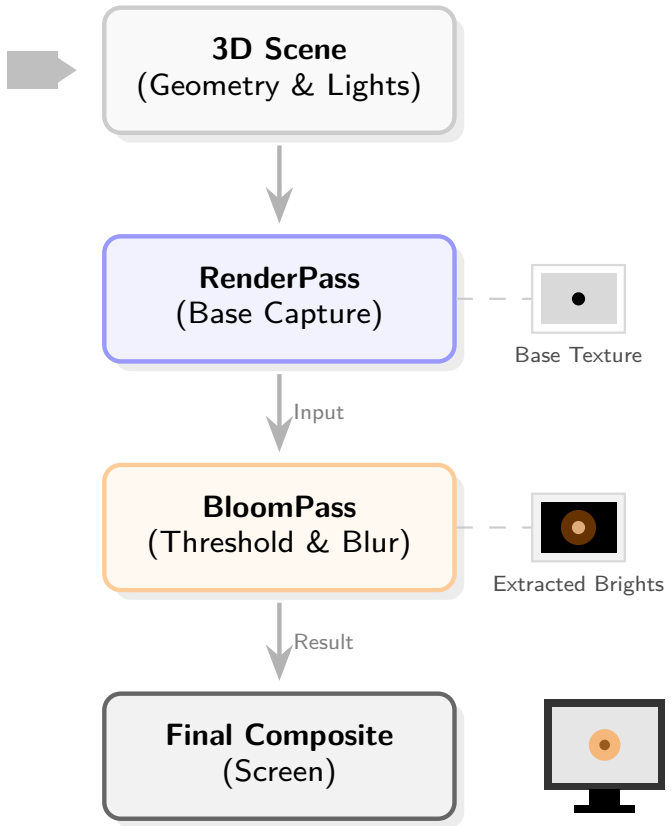
```
// 1. IMPORTS (Required Extras)
import { EffectComposer } from '.../EffectComposer.js';
import { RenderPass } from '.../RenderPass.js';
import { UnrealBloomPass } from '.../UnrealBloomPass.js';

// 2. SETUP COMPOSER
// This replaces the default renderer
const composer = new EffectComposer( renderer );

// 3. ADD PASS 1: Base Scene
// "Take the 3D scene and camera, render to texture"
const renderPass = new RenderPass( scene, camera );
composer.addPass( renderPass );

// 4. ADD PASS 2: Bloom Effect
// Params: resolution, strength, radius, threshold
const bloomPass = new UnrealBloomPass(
    new THREE.Vector2( w, h ),
    1.5,  // Strength (Intensity)
    0.4,  // Radius (Spread)
    0.85  // Threshold (Only bright pixels glow)
);
composer.addPass( bloomPass );

// 5. RENDER LOOP
function animate() {
    // renderer.render( scene, camera ); // <-- OLD WAY
    composer.render();                   // <-- NEW WAY
}
```



How it works

The `UnrealBloomPass` automatically isolates pixels brighter than the `threshold`, blurs them, and adds them back on top of the original image.